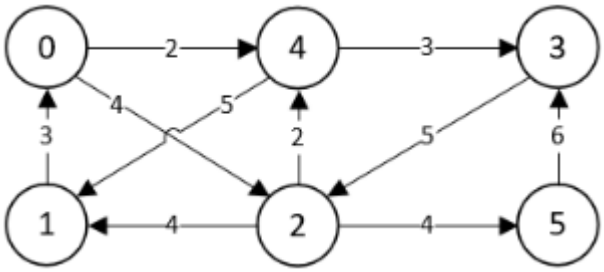
图 作业

单选题 9 (总分：9.00)

#	题目	分值
1	使用Dijkstra算法求下图中从顶点0到其他各顶点的最短路径，依次得到的各最短路径的目标顶点是 _____ 	1.00

- A. 2,4,1,3,5
- B. 4,2,1,5,3
- C. 4,3,2,1,5
- D. 4,2,3,1,5

正确答案：D

答案解析：

0-1 7

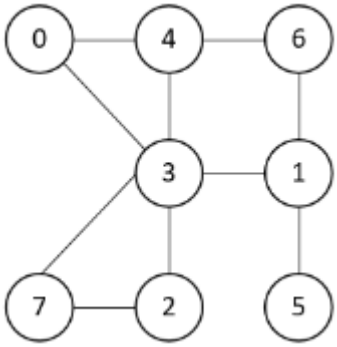
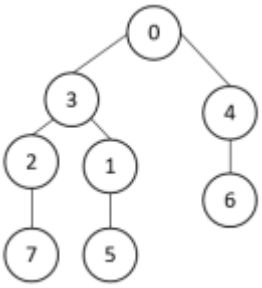
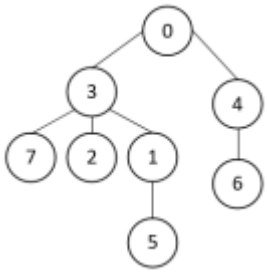
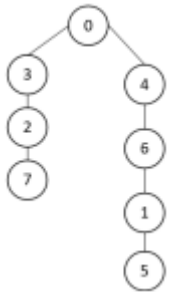
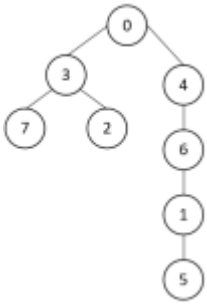
0-2 4

0-3 5

0-4 2

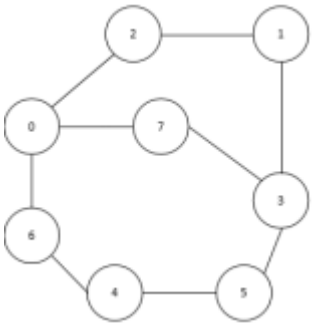
0-5 8

从小到大进行排序

#	题目	分值
2	 对上图进行BFS遍历，记录遍历过程即可得到图的生成树 _____ ○ A.  ○ B.  ○ C.  ○ D.  正确答案：B	1.00

#	题目	分值

3 1.00



对上面的无向图进行深度优先遍历，从0开始，可能的访问顺序为 _____

○ A. 0-2-1-3-5-4-6-7

○ B. 0-1-2-3-4-5-6-7

○ C. 0-6-4-5-3-2-1-7

○ D. 0-2-1-7-3-6-4-5

正确答案：A

4 某有向图的顶点数和边数目都为4，建立链表总共需要 _____个链表结点 1.00

○ A. 4

○ B. 8

○ C. 12

○ D. 16

正确答案：B

答案解析：

某有向图的顶点数和边数目分别为n和e，建立链表总共需要(n+e)个链表结点

5 一个包含有24个结点的有向图，最多包含多少条边 _____ 1.00

○A. 276

○B. 552

○C. 562

○D. 576

正确答案: B

答案解析:

含有n个结点的有向图，最多含有n（n-1）条边

#	题目	分值
6	一个有n个顶点和n条边的无向图一定是 _____。	1.00

○ A. 连通的
○ B. 不连通的
○ C. 无环的
○ D. 有环的

正确答案: D

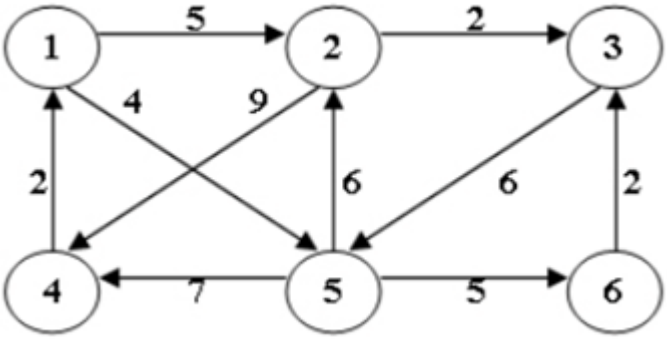
答案解析:

一个有n个顶点和n条边的无向图可能是连通的，也可能是非连通的。这取决于边的分布情况。如果这n条边能够将所有n个顶点连接起来，那么这个图就是连通的；反之，如果边的分布导致某些顶点无法互相到达，那么这个图就是非连通的。

此外，由于无向图中每条边连接两个顶点，所以n条边必然连接n个顶点，且每个顶点的度（连接到该顶点的边的数量）至少为1。在这种情况下，如果每条边都连接不同的顶点对，那么这个图将形成一个环，即每个顶点都与除了自身以外的其他顶点相连，形成一个闭环。因此，一个有n个顶点和n条边的无向图一定是有环的。

综上所述，正确答案是 D. 有环的。

7	下图中从顶点1到其他各顶点的最短路径，从短到长依次为 _____。	1.00



○ A. 5, 2, 3, 4, 6
○ B. 5, 2, 3, 6, 4
○ C. 5, 2, 4, 3, 6
○ D. 5, 2, 6, 3, 4

正确答案: B

答案解析:

1→5:4

1→2:5

1→3:7

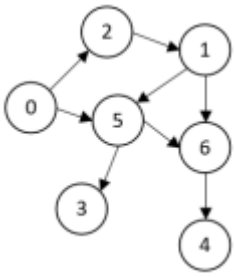
1→6:9

1→4:11

#	题目	分值
8	图的广度优先遍历类似于二叉树的 _____。 ○A. 先序遍历 ○B. 中序遍历 ○C. 后序遍历 ○D. 层次遍历 **正确答案:** D	1.00
9	在以下关于最短路径算法的描述中，哪个是正确的？ _____ ○A. Dijkstra算法能在包含负权边的图中找到最短路径，但不能再包含负权环的图中找到最短路径。 ○B. Bellman-Ford算法的时间复杂度是O(V*E)，其中V是顶点数，E是边数。 ○C. Dijkstra算法只能用于有向图。 ○D. Bellman-Ford算法可以处理包含负权环的图。 **正确答案:** B **答案解析:** 选项B是正确的。 A. Dijkstra算法不能在包含负权边的图中找到最短路径，因为它基于这样的假设：一旦找到从源点到某个顶点v的最短路径，那么这个最短路径也将是最终的最短路径。负权边可能导致在后续的迭代中发现到达同一个顶点v的更短路径，这与Dijkstra算法的基本原则相违背。因此，如果图中包含负权边，就不能使用Dijkstra算法。同时，Dijkstra算法也不能处理包含负权环的图。 B. Bellman-Ford算法的时间复杂度确实是O(V*E)，因为它涉及到对所有的边进行松弛操作，这样的操作需要进行V-1次，其中V是顶点数，E是边数。 C. Dijkstra算法不仅可以用于有向图，也可以用于无向图。只要图中没有负权边和负权环，Dijkstra算法就能找到从单一源点到其他所有顶点的最短路径。 D. Bellman-Ford算法可以处理包含负权边的图，但它不能处理包含负权环的图。如果图中存在负权环，那么Bellman-Ford算法可能会进入无限循环，因为它会不断发现更短的路径。在实际应用中，如果图中存在负权环，那么最短路径问题是没有意义的，因为可以通过负权环无限降低路径长度。	1.00

填空题 12 （总分：12.00）

#	题目	分值
1	Floyd算法如果采用一个路径矩阵来保存最短路径的顶点序列，其时间复杂度为O(_____)，空间复杂度为O(_____) **正确答案:** n^3 | n^2	1.00

#	题目	分值
2	 对上面的有向图进行广度优先遍历，从0开始，最后访问的节点为： _____ **正确答案:** 4	1.00
3	某无向图的顶点数和边数目分别为5和6，建立链表总共需要 _____ 个链表结点 **正确答案:** 17 **答案解析:** 若无向图的顶点数和边数目分别为n和e，总共需要(n+2e)个链表结点	1.00
4	无向连通图G有欧拉路径当且仅当G仅有_____个或者_____个顶点度为奇数 **正确答案:** 2 或 两 | 0 或 零	1.00
5	具有10个顶点的无向完全图，其边的个数为 _____ 。 **正确答案:** 45 **答案解析:** $e=n\times(n-1)/2$	1.00
6	拥有4个顶点的无向图共有 _____ 种，有向图共有 _____ 种。 **正确答案:** 64 或 2^6 | 4096 或 2^12 **答案解析:** 拥有n个顶点的无向图共有 2^{n(n-1)/2}种，有向图共有2^{n(n-1)}种。	1.00
7	在无向图中，如果从任一顶点出发都可以通过遍历访问到其他所有顶点，则称该图为 _____ 图。在有向图中，如果对于任意的顶点 u 和 v，都存在从 u 到 v 和从 v 到 u 的路径，则该图被称为 _____ 图。 **正确答案:** 连通 | 强连通	1.00
8	在图的最短路径问题中， _____ 算法用于计算从单一源点出发到所有其他节点的最短路径，其采用的数据结构通常是 _____ 。 **正确答案:** Dijkstra | 优先队列 或 最小堆	1.00

#	题目	分值
9	Bellman-Ford算法的基本思想是通过 _____ 操作来逐步逼近最短路径，与Dijkstra算法相比其优势是可以处理含 _____ 的图。 **正确答案:** 松弛 或 Relaxation | 负权边 或 负值边	1.00
10	在Bellman-Ford算法中，对于含有n个顶点和m条边的带权有向图G=(V,E)，若从源点s出发进行松弛操作，至少需要经过 _____ 次迭代，才能确保图中所有顶点的距离估计值均达到最小。 **正确答案:** n-1	1.00
11	对于一个含有n个顶点的带权无向图，其最小生成树的边数一定为 _____ . **正确答案:** n-1	1.00
12	邻接矩阵、邻接表都是常用的图表征方式，其中稠密图适于采用 _____ 表示，大规模的稀疏图适于采用 _____ 表示。 **正确答案:** 邻接矩阵 | 邻接表	1.00

简答题 3 （总分：16.00）

#	题目	分值
1	**最短路径** 现有一个存在负权边的图G1，是否可以将图G1的每条边的权加上一个常数的到新图G2，使新图G2中无负权边，然后对新图G2利用Dijkstra算法求原图G1最短路径，为什么？ **参考答案:** 不行，因为对新图G2利用Dijkstra算法求解最短路劲会加重多跳的路径的权重，从而导致计算出的最短路径与原图G1中的最短路径不同。	5.00
2	**最短路径** 为什么Kruskal算法得到的一定是最小生成树? **参考答案:** 由于Kruskal算法每一步都是基于局部最优解（即当前可用的最小权重边）进行选择，并且通过并查集数据结构有效避免了环的形成，因此可以保证全局最优解，即得到图的最小生成树。Kruskal算法的正确性基于以下两个重要性质：贪心选择的性质：Kruskal算法按照边的权重进行贪心选择，每一步都选择当前可用的最小权重边。这种选择策略保证了最终得到的边的集合是权重最小的。切分（Cut）：在算法的任何阶段，假设已经选择了一些边，这些边将图的所有顶点分为两个集合。选择那些连接这两个集合的、权重最小的边不会形成环，因为这样的边只会将两个分离的集合连接起来，而不会在已连接的顶点内部形成新的连接。	5.00

#	题目	分值
3	**无向图** 为什么无向连通图的顶点度都为偶数时，则该图没有桥？ **参考答案:** 顶点度均为偶数的无向连通图存在欧拉回路，因此删除任何一条边不影响连通性。或者删除一条边之后，该边关联的两个顶点的度变为奇数，分属到两个子图中会导致违反握手原理。	6.00

编程题 1 （总分：10.00）

#	题目	分值
1	**最短通勤时间**	10.00

【问题描述】

假设存在一个地图, 上面标有N个编号独一无二的城市(编号从0到N-1)。现在, 一名旅行者计划从城市0出发, 乘坐火车前往城市N-1。需要注意的是, 并非所有城市之间都设有直达火车, 因此, 每当旅行者需要换乘时, 都需要额外增加H小时的通勤时间。请设计一个程序, 能够计算出从城市0到城市N的最短通勤时间。

【输入形式】

第一行包含三个整数N、M、H, 分别表示图中的城市数, 存在直达火车的城市对数，换乘时间。接下来M行, 每行包含三个整数a, b, w，表示存在从城市a到城市b的直达火车, 通勤时间为w。

【输出形式】

最短通勤时间, 如果不存在连通路径则输出inf

【样例输入】

5 7 1

0 1 2

0 2 3

1 2 3

1 3 5

2 4 3

3 2 4

3 4 4

【样例输出】

7

【样例说明】

第一行的三个整数5 7 1, 分别表示城市数为5, 存在直达火车的城市对数为7，换乘时间为1小时。接下来7行, 每行包含三个整数a, b, w，表示存在从城市a到城市b的直达火车, 通勤时间为w，如接下来的第一行0 1 2表示存在从城市0到城市1的直达火车, 通勤时间为2小时。输出7表示从城市0到城市N的最短通勤时间为7小时。

【评分标准】

参考答案:

#	题目	分值

```
import heapq
import random


def dijkstra(graph, start):
    n = len(graph)
    distances = [float('inf')] * n
    distances[start] = 0
    visited = set()

    priority_queue = [(0, start)]

    while priority_queue:
        current_distance, current_node = heapq.heappop(priority_queue)

        if current_node in visited:
            continue

        visited.add(current_node)

        for neighbor, weight in graph[current_node].items():
            distance_through_current = current_distance + weight

            if distance_through_current < distances[neighbor]:
                distances[neighbor] = distance_through_current
                heapq.heappush(priority_queue, (distance_through_current, nei
ghbor))

    return distances[-1]

def main():
    n, m, h = map(int, input().split())
    graph = [[] for _ in range(n)]

    for _ in range(m):
        u, v, w = map(int, input().split())
        graph[u].append((v, w + h))

    for node in range(n):
        graph[node] = dict(graph[node])

    result = dijkstra(graph, 0)
    print(result - h)

if __name__ == '__main__':
    main()
```